Reviewer Pack — Noncommutative L^p Norm Lower Bound for Disjointness Communi…

Entry 5958cc0b1ee5 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 10:44:31 UTC

Noncommutative L^p Norm Lower Bound for Disjointness Communication Complexity

Entry ID: `5958cc0b1ee5` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 10:44:31 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative L^p Geometry **Field B** (complexity object): Communication Complexity of Disjointness

Statement:

> For a random n×n matrix M with entries 1 iff i ∩ j = □, the noncommutative L^p norm ||M||_p ≥ Ω(n^{1/2}) for p ∈ (1,2). This bound is tight up to polylog(n) factors.

Rationale (proposer's reasoning):

> Noncommutative L^p spaces capture the 'geometric complexity' of matrix entries under tensor products, which mirrors the entanglement structure in multi-party communication. The p=2 case corresponds to operator norms, while p→1 recovers the max-entropy of disjointness witnesses.

Taxonomy category: `BOUNDED_ARITHMETIC` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `27a6e580a39f9c19`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

> default: >=80% seeds must support, no counterexample

3. Barrier filter (F1)

Final: `PASS`

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.95	UNCERTAIN	SAFE
ALGEBRIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM_1	LLM_2
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    n = 40
    p_values = [1.5, 2.0, 2.5]

    def generate_disjointness_matrix(n):
        M = [[0] * n for _ in range(n)]
        for i in range(n):
            for j in range(i + 1, n):
                if random.choice([True, False]):
                    M[i][j] = 1
                    M[j][i] = 1
        return M

    def singular_values(M):
        # Compute the singular values of a matrix using power iteration method
        U, S, Vt = [], [], []
        A = M
        for _ in range(100):  # Power iteration iterations
            v = [random.gauss(0, 1) for _ in range(n)]
            u = [sum(A[i][j] * v[j] for j in range(n)) for i in range(n)]
            u_norm = math.sqrt(sum(x**2 for x in u))
            u = [x / u_norm for x in u]

            s = sum(u[i] * A[i][j] * v[j] for i in range(n) for j in range(n))
            S.append(s)

            v = [sum(A[j][i] * u[j] for j in range(n)) for i in range(n)]
            v_norm = math.sqrt(sum(x**2 for x in v))
            v = [x / v_norm for x in v]

        return sorted(S, reverse=True)

    def noncommutative_lp_norm(singular_values, p):
        return sum(s**(p/(p-1)) for s in singular_values)**(1/p)

    min_norm = float('inf')
```

```
    for _ in range(30):  # Number of instances per seed
        M = generate_disjointness_matrix(n)
        norm = noncommutative_lp_norm(singular_values(M), p)
        if norm < min_norm:
            min_norm = norm

    return {
        "metric_name": "noncommutative lp norm",
        "metric_value": min_norm,
        "instances_tested": 30,
        "conjecture_holds": min_norm >= 0.1 * math.sqrt(n),
        "counterexample": "" if min_norm >= 0.1 * math.sqrt(n) else f"min_norm={min_norm}, n={n}"
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        random.seed(seed)
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(result["metric_value"] for result in results) / len(results)
    std_value = math.sqrt(sum((result["metric_value"] - mean_value)**2 for result in results) / le
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std={std_value} support_fraction={support_frac
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9e52a4c.py", line 74, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c9e52a4c.py", line 56, in run_trial
    norm = noncommutative_lp_norm(singular_values(M), p)
                                                      ^
NameError: name 'p' is not defined
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> Test crashed due to undefined variable 'p', preventing data collection. Pre-registered support condition cannot be evaluated without valid trial results. | next: Fix the NameError in test_c9e52a4c.py by ensuring 'p' is properly defined in the noncommutative_lp_norm function call

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	83015
2	propose	ollama_remote	qwen3:8b	0	0	115007
3	propose	ollama_remote	qwen3:8b	0	0	35526
4	propose	ollama_remote	qwen3:8b	0	0	48452
5	preregistration	ollama_remote	qwen3:8b	0	0	27450
6	novelty	ollama_remote	qwen3:8b	0	0	24157
7	novelty	ollama_remote	qwen3:8b	0	0	17068
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11111
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9204
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9385
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10071
12	judge	ollama_remote	qwen3:8b	0	0	20047

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 410492 ms total latency. Provider mix: {'ollama_remote': 12}

(full prompt+response transcripts available in `research/audit/5958cc0b1ee5.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/5958cc0b1ee5.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/5958cc0b1ee5.tar.gz` (if generated)